

Seven Seas A-53

Security Audit

July 21, 2025

Version 1.0.0

Table of Contents

- [Introduction](#)
- [Overall Assessment](#)
- [Specification](#)
- [Source Code](#)
- [Issue Descriptions and Recommendations](#)
- [Security Levels Reference](#)
- [Issue Details](#)
- [Disclaimer](#)

Introduction

This document includes the results of the security audit for Seven Seas's smart contract code as found in the section titled 'Source Code'. The security assessment of TAC bridging was performed by the Macro security team on July 10th 2025.

The purpose of this audit is to review the source code of certain Seven Seas Solidity contracts, and provide feedback on the design, architecture, and quality of the source code with an emphasis on validating the correctness and security of the software in its entirety.

Disclaimer: While Macro's review is comprehensive and has surfaced some changes that should be made to the source code, this audit should not solely be relied upon for security, as no single audit is guaranteed to catch all possible bugs.

Overall Assessment

The following is an aggregation of issues found by the Macro Audit team:

Severity	Count	Acknowledged	Won't Do	Addressed
Medium	2	-	-	2
Low	1	-	-	1

Seven Seas was quick to respond to these issues.

Specification

Our understanding of the specification was based on the following sources:

Security Analysis

TAC CrossChainLayer bridge

- General
 - Documentation: No available documentation
 - A bridging protocol on the TAC chain, allowing bridging to and from the TON network
 - Main components are:
 - Cross Chain Layer: Contract that allows sending and receiving messages to and from TON.
- Security Evaluation

- Audited by Trail of Bits

Although it is a fairly standard bridging contract, it is upgradeable and is setup to allow for additional message types in future upgrades. It has minimal owner privileged functions allowing a protocol fee to be set, a tokenUtils contract to be initialized, and protocol fees to be withdrawn, all fairly standard functions where the only concern would be a unexpected raise to the protocol fee, or a malicious upgrade that could steal locked assets. The owner is [currently a 2:2 safe](#) contract without a timelock, which is better than an EOA, but should be watched for malicious activity before interacting with the bridge.

This has been audited by a notable and trusted auditing firm, Trail of Bits, and all issues found have been resolved, giving confidence that there are no lingering issues.

It also should be noted that interacting with bridges requires trust that a call on the destination chain are made as expected, so interacting with bridges in general has some inherent risk.

Rating: C (Upgradeable with semi-secure ownership, bridge protocols are inherently risky and require outside trust)

TAC intermediate USDT bridging contracts on TON

- General
 - Documentation: No available documentation
 - An intermediate bridging protocol on the TON chain, intended to be used to bridge USDT from TAC to ETH or ETH to TAC by first receiving a bridge request and forwarding USDT to the appropriate chain using Layer 0 when interacting with ETH mainnet and the Cross Chain Layer when interacting with TAC.
 - Main components are:

- ETH USDT Treasury: Contract that sends USDT received to ETH mainnet via layer 0.
- TAC USDT Treasury: Contract that sends USDT received to TAC via the cross chain layer bridge
- Security Evaluation
 - Audited by Quantstamp
 - Contracts:
 - [tac_usdt_treasury](#)
 - [eth_usdt_treasury](#)

Straight forward bridging forwarder contract. Notably the 3 issues found by Quantstamp have not been resolved but they are minor and should not effect user interactions.

Rating: B (bridge protocols are inherently risky and require outside trust)

Source Code

The following source code was reviewed during the audit:

Specifically, we conducted a security review based the following TAC and TON contracts:

- [CrossChainLayer](#): 0x9fee01e948353E0897968A3ea955815aaA49f58d
- [Implementation](#): 0xDd57514b2Bee6b4575651cB58c68B390815589e9
- [tac_usdt_treasury](#)
- [eth_usdt_treasury](#)
- Additionally, we audited the associated TACTON decoder and Sanitiser (from [PR 392](#))

Source Code	SHA256
src/base/DecodersAndSanitizers/TacTON DecoderAndSanitizer.sol	b160b73b8f593dfc7bdd7c36a5f664d2 fbc1811b3f01838bd730d57093b6704f

Note: This document contains an audit solely of the Solidity contracts listed above. Specifically, the audit pertains only to the contracts themselves, and does not pertain to any other programs or scripts, including deployment scripts.

Issue Descriptions and Recommendations

Click on an issue to jump to it, or scroll down to see them all.

M-1 Missing sanitization checks allow bridging any NFT

M-2 Incomplete decoding of `tvmTarget` address

L-1 Constrain message version

Security Level Reference

We quantify issues in three parts:

1. The high/medium/low/spec-breaking **impact** of the issue:

- How bad things can get (for a vulnerability)
- The significance of an improvement (for a code quality issue)
- The amount of gas saved (for a gas optimization)

2. The high/medium/low **likelihood** of the issue:

- How likely is the issue to occur (for a vulnerability)

3. The overall critical/high/medium/low **severity** of the issue.

This third part – the severity level – is a summary of how much consideration the client should give to fixing the issue. We assign severity according to the table of guidelines below:

Severity	Description
(C-x) Critical	We recommend the client must fix the issue, no matter what, because not fixing would mean significant funds/assets WILL be lost.
(H-x) High	We recommend the client must address the issue, no matter what, because not fixing would be very bad, or some funds/assets will be lost, or the code's behavior is against the provided spec.
(M-x) Medium	We recommend the client to seriously consider fixing the issue, as the implications of not fixing the issue are severe enough to impact the project significantly, albeit not in an existential manner.
(L-x) Low	The risk is small, unlikely, or may not be relevant to the project in a meaningful way. Whether or not the project wants to develop a fix is up to the goals and needs of the project.
(Q-x) Code Quality	The issue identified does not pose any obvious risk, but fixing could improve overall code quality, on-chain composability, developer ergonomics, or even certain aspects of protocol design.
(I-x) Informational	Warnings and things to keep in mind when operating the protocol. No immediate action required.
(G-x) Gas Optimizations	The presented optimization suggestion would save an amount of gas significant enough, in our opinion, to be worth the development cost of implementing it.

Issue Details

M-1

Missing sanitization checks allow bridging any NFT

TOPIC	STATUS	IMPACT	LIKELIHOOD
Sanitization	Fixed ↗	Medium	Low

[TacCrossChainLayerDecoderAndSanitizer](#) is a decoder for the Cross Chain layer bridging contract on the TAC network. The current intended use case is to transfer USDT to and from TAC to ETH mainnet. There is no current intention to bridge NFTs, however [sendMessage\(\)](#) allows for NFTs to be bridged as defined by the encoded [NFTAmount](#) parameter. This value is not sanitized, allowing any value to be used in the NFTAmount array, resulting in allowing strategists to bridge any supported NFT without merkle tree authorization.

Remediations to Consider

Add a check that reverts if the NFTAmount array length is nonzero.

M-2

Incomplete decoding of `tvmTarget` address

TOPIC	STATUS	IMPACT	LIKELIHOOD
Sanitation	Fixed ↗	Low	Low

In [sendMessage\(\)](#), the `tvmTarget` is decoded from a string of arbitrary size into 2 separate addresses to cover the larger sized addresses used on the TON network:

```

//convert ton address to bytes
bytes memory tvMBytes = bytes(messageV1.tvMTarget);

//sanity check
if (tvMBytes.length < 20) revert TacCrossChainLayerDecoderAndSanitizer__7

address tvMTarget0;
assembly {
    tvMTarget0 := mload(add(tvMBytes, 32))
}

// Extract second address (bytes 20–39) if available
address tvMTarget1;
if (tvMBytes.length >= 40) {
    assembly {
        tvMTarget1 := mload(add(tvMBytes, 52)) // skip length prefix + 20
    }
}

```

Reference: [TacCrossChainLayerDecoderAndSanitizer.sol#L18-35](#)

However TON addresses are 48 bytes, larger than the 40 bytes covered by decoding 2 addresses, and the first address decoding at location 32 cuts off the first 12 bytes when it gets cast from bytes32 to an address, resulting in an incomplete decoding of the target address, allowing bridging to an unintended address that matches the last 36 bytes.

Remediations to Consider

Use 3 addresses, loading from memory location increments of 20, 40, and 60.



Constrain message version

TOPIC

Input Validation

STATUS

Fixed [↗](#)

IMPACT

Medium

LIKELIHOOD

Low

The [Cross chain layer contract](#) currently only allows for one version of message, however it is upgradeable and setup to be upgraded to include additional message types which is specified by the `messageVersion` parameter. The decoder assumes the message is version 1, and decodes it as if it is, however if there are additional message versions added in the future, a differing version number could be called and the decoder would decode the message improperly, likely resulting in an error but could allow for unexpected and unverified actions to be made if verification passes.

Remediations to Consider

Constrain the version number to be equal to 1.

Disclaimer

Macro makes no warranties, either express, implied, statutory, or otherwise, with respect to the services or deliverables provided in this report, and Macro specifically disclaims all implied warranties of merchantability, fitness for a particular purpose, noninfringement and those arising from a course of dealing, usage or trade with respect thereto, and all such warranties are hereby excluded to the fullest extent permitted by law.

Macro will not be liable for any lost profits, business, contracts, revenue, goodwill, production, anticipated savings, loss of data, or costs of procurement of substitute goods or services or for any claim or demand by any other party. In no event will Macro be liable for consequential, incidental, special, indirect, or exemplary damages arising out of this agreement or any work statement, however caused and (to the fullest extent permitted by law) under any theory of liability (including negligence), even if Macro has been advised of the possibility of such damages.

The scope of this report and review is limited to a review of only the code presented by the Seven Seas team and only the source code Macro notes as being within the scope of Macro's review within this report. This report does not include an audit of the deployment scripts used to deploy the Solidity contracts in the repository corresponding to this audit. Specifically, for the avoidance of doubt, this report does not constitute investment advice, is not intended to be relied upon as investment advice, is not an endorsement of this project or team, and it is not a guarantee as to the absolute security of the project. In this report you may through hypertext or other computer links, gain access to websites operated by persons other than Macro. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such websites' owners. You agree that Macro is not responsible for the content or operation of such websites, and that Macro shall have no liability to your or any other person or entity for the use of third party websites. Macro assumes no responsibility for the use of third party software and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any outcome generated by such software.